

Data Engineering - Data Foundry Challenge

Case Description

Build a **data pipeline** that ingests and enriches documents from [Domínio Público](#), Brazil's public domain digital library.

You are given building block scripts that handle scraping, hashing, LLM descriptions, translations, and cover extraction. Your job is to orchestrate them into a pipeline and produce two output datasets.

You will be evaluated on pipeline design and engineering decisions, not on the scripts themselves.

What You're Given

- **8 scripts** in `src/data_foundry/scripts/` (download, hash, describe, translate, covers, output assembly). These are tools, not the solution. Use, modify, or rewrite as needed.
- **Infrastructure:** `Dockerfile`, `compose.yaml` (Ollama), `Makefile`, `.env.example`
- **LLM integration** via OpenAI-compatible API, swappable through env vars (`LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`). Feel free to change to any other models or providers.
- **Tests** in `tests/test_outputs.py` validating the final outputs

Minimum scope: Process at least **10 books** end-to-end. You are free to expand beyond 10 to demonstrate scale.

What You Must Deliver

We define 5 target areas. **Choose at least 2** to tackle in your delivery:

1. Event-Driven Pipeline

Stages react to data availability, not hardcoded sequencing. Message queues, file watchers, workflow engines, or any event-driven pattern.

2. Data Architecture

Separate raw and processed data into distinct layers. Define clear schemas for the two output datasets. Justify additional layers if you add them.

3. Versioning

Re-runs don't overwrite previous outputs. Timestamps, run IDs, snapshots, or any approach you choose.

4. Scalability

The pipeline handles larger data volumes. Parallel processing, rate limiting, backpressure, pagination, or any approach you choose.

5. Data Quality

Tackle problems like missing data, encoding issues, duplicates, and other issues that make the solution more robust.

Output Datasets

Your pipeline must produce two JSON files:

`localized_catalog.json` (language-dependent):

- Work identifier, author
- Title and description in PT, EN, ES, FR

`universal_metadata.json` (language-independent):

- Work identifier, document hash, cover path
- Access count, file size, category, year

Both files can be expanded if desired.

How to Run

```
make run # run pipeline end-to-end via Docker
make test # validate outputs
```

Individual steps available via `make download`, `make hash`, `make describe`, etc.

Documentation

Your README must include:

- Setup instructions (`make run` must work)
- Architecture description or diagram
- Design decisions and trade-offs

Deliverables

- Public GitHub repository with full implementation
- README with setup instructions and usage examples
- `.env.example` and any required secrets/configs

Good luck!